Rapport d'equipe
Ajout d'une nouvelle serie STM32Cube dans le pipeline

Projet STM32Cube Preprocessing

28 juillet 2026

Table des matières

1 Objet du rapport **2**

2 Perimetre technique **2**

3 Fichiers a modifier **2**
- 3.1 Configuration . . . 2
- 3.2 Scripts workflow . . . 2
- 3.3 Workflows CI/CD . . . 2

4 Definition des champs de configuration **2**
- 4.1 Fichier `config_<serie>.json` . . . 2
- 4.2 Fichier `config_all_series.json` . . . 3

5 Source des informations : repos, urls, refs **3**
- 5.1 Commandes de collecte . . . 3
- 5.2 Regles de remplissage . . . 3

6 Procedure d'integration d'une nouvelle serie **4**
- 6.1 Etape 1 : creation du fichier config serie . . . 4
- 6.2 Etape 2 : declaration globale . . . 4
- 6.3 Etape 3 : mapping scripts . . . 4
- 6.4 Etape 4 : mapping batch Python . . . 4
- 6.5 Etape 5 : mapping GitHub Actions . . . 4

7 Validation fonctionnelle **4**
- 7.1 Prepare . . . 4
- 7.2 Split composant . . . 4
- 7.3 Controle couverture et doublons . . . 4
- 7.4 Upload composant . . . 5

8 Test sur une autre machine (self-hosted runner) **5**
- 8.1 Prerequis . . . 5
- 8.2 Installation projet . . . 5
- 8.3 Runner et configuration . . . 5
- 8.4 Test de bout en bout . . . 5

9 Mode migration vers datasources par composant **5**

10 Checklist Go/No-Go **6**

11 Points d'attention **6**

1 Objet du rapport

Ce document formalise la procedure standard d'ajout d'une nouvelle serie STM32Cube (exemple : C5) dans le pipeline. Il precise les fichiers a modifier, la signification des champs de configuration, la source des donnees a collecter, l'integration dans l'automatisation, ainsi que les etapes de validation avant mise en production.

2 Perimetre technique

Le perimetre couvre :

— la configuration serie,

— l'execution du pipeline (Prepare/Upload),

— le split des issues par composant,

— l'upload vers la KB,

— l'integration dans les workflows GitHub Actions,

— le test sur une autre machine avec self-hosted runner.

3 Fichiers a modifier

3.1 Configuration

— `shared/config/config_c5.json` (nouveau fichier)

— `shared/config/config_all_series.json` (mise a jour)

3.2 Scripts workflow

— `pipeline_Automation/workflow/Run_Single_Series_Full_Pipeline_And_Upload.ps1`

— `pipeline_Automation/workflow/Split_Issues_By_Component.ps1`

— `pipeline_Automation/workflow/Upload_Issues_By_Component.ps1`

— `pipeline_Automation/workflow/run_all_series.py`

3.3 Workflows CI/CD

— `.github/workflows/auto_update_kb.yml`

— `.github/workflows/manual_auto_update_kb.yml`

4 Definition des champs de configuration

4.1 Fichier `config_<serie>.json`

owner

Organisation source des depots (STMicroelectronics).

repos

Liste ordonnee des depots de la serie. Le premier element doit etre le repo parent (exemple : STM32CubeC5), suivi des depots lies (HAL/CMSIS/BSP/Middleware).

data_dir

Dossier des artefacts data intermediaires (valeur standard : `data`).

repos_dir

Dossier local des clones (valeur standard : `GitHub_repos`).

auto_clone_missing_repos

Activation du clonage automatique des depots manquants.

repo_clone_urls
Dictionnaire `repo -> URL git` pour les depots relies.

linked_repo_refs
Dictionnaire `repo -> SHA` correspondant aux refs des sous-modules pour la release cible. Ce champ est essentiel pour la reproductibilite.

commit_pr_scope
Strategie de liaison des commits/PR (valeur recommandee : `linked_or_merged`).

github_token_env
Nom de variable d'environnement contenant le token GitHub.

rate_limits
Temporisations API (issues/comments) pour stabiliser les appels.

ingest_version
Version logique de la configuration d'ingestion.

4.2 Fichier `config_all_series.json`

all_series
Liste des series actives.

series_config_files
Mapping `Serie -> fichier config`.

kb_datasource_ids
IDs des datasources par serie (issues, files, diagnostic, resolver).

kb_id
Identifiant de la Knowledge Base cible.

5 Source des informations : repos, urls, refs

Les informations sont extraites du repo parent de la serie, positionne sur la release cible.

5.1 Commandes de collecte

```
git clone --recursive https://github.com/STMicroelectronics/STM32CubeC5.git
cd STM32CubeC5
git checkout <tag_cible>
git submodule update --init --recursive

# URLs des sous-modules
git config -f .gitmodules --get-regexp "submodule\..*\.url"

# SHAs de references
git submodule status --recursive

# Paths des sous-modules
git config -f .gitmodules --get-regexp "submodule\..*\.path"
```

5.2 Regles de remplissage

— `repos[0]` = repo parent de la serie.

— `repo_clone_urls` renseigne les depots a cloner.

— `linked_repo_refs` renseigne les SHAs de la release cible.

— Exclure les depots non relies au perimetre de la serie.

6 Procedure d'integration d'une nouvelle serie

6.1 Etape 1 : creation du fichier config serie

Creer `shared/config/config_c5.json` a partir d'un modele proche (`config_c0.json` ou `config_g4.json`) et renseigner les champs `repos`, `repo_clone_urls`, `linked_repo_refs`.

6.2 Etape 2 : declaration globale

Mettre a jour `shared/config/config_all_series.json` :

— ajout dans `all_series`,

— mapping dans `series_config_files`,

— entree dans `kb_datasource_ids`.

6.3 Etape 3 : mapping scripts

Ajouter la serie dans tous les `ValidateSet` et `seriesMap` :

— code serie : `C5`

— slug serie : `stm32cubec5`

— repo parent : `STM32CubeC5`

— config : `shared/config/config_c5.json`

6.4 Etape 4 : mapping batch Python

Dans `run_all_series.py`, ajouter `c5` dans `SERIES_CONFIGS`.

6.5 Etape 5 : mapping GitHub Actions

Ajouter la serie C5 dans :

— les listes de series autorisees,

— les listes par defaut (si desired),

— les normalisations d'inputs.

7 Validation fonctionnelle

7.1 Prepare

```
powershell -ExecutionPolicy Bypass -File .\pipeline_Automation\workflow\
    Run_Single_Series_Full_Pipeline_And_Upload.ps1 -Series C5 -Mode Prepare
```

7.2 Split composant

```
powershell -ExecutionPolicy Bypass -File .\pipeline_Automation\workflow\
    Split_Issues_By_Component.ps1 -Series C5
```

7.3 Controle couverture et doublons

```
python -m pipeline.delivery.validate_issues_by_component_split --summary "datasets/07
    _delivery/st_ready/by_series/stm32cubec5/issues_json/by_component/
    summary_by_component_c5.json"
```

7.4 Upload composant

```
powershell -ExecutionPolicy Bypass -File .\pipeline_Automation\workflow\
    Upload_Issues_By_Component.ps1 -Series C5 -DryRun -RemoteUser "<email_st>"
powershell -ExecutionPolicy Bypass -File .\pipeline_Automation\workflow\
    Upload_Issues_By_Component.ps1 -Series C5 -RemoteUser "<email_st>"
```

8 Test sur une autre machine (self-hosted runner)

8.1 Prerequis

— Windows 10/11

— Git

— Python 3.12+

— acces reseau GitHub/ST

8.2 Installation projet

```
git clone https://github.com/<org-ou-user>/PFE_Chatbot_STM32Cube.git
cd PFE_Chatbot_STM32Cube
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

8.3 Runner et configuration

— Declarer un self-hosted runner depuis les settings du repo.

— Lancer le runner en service.

— Verifier le label `self-hosted`.

— Activer `git core.longpaths=true`.

— Configurer secrets : `ST_CHATGPT_API_KEY`, `ST_REMOTE_USER`.

8.4 Test de bout en bout

```
powershell -ExecutionPolicy Bypass -File .\pipeline_Automation\workflow\
    Run_Single_Series_Full_Pipeline_And_Upload.ps1 -Series C5 -Mode Full -
    EnableIssuesByComponentFlow -ComponentIssuesOnly -RemoteUser "<email_st>"
```

9 Mode migration vers datasources par composant

Pour remplacer le datasource global issues par des datasources `issues_<SERIE>_<COMPONENT>` :

— activer `EnableIssuesByComponentFlow`,

— activer `ComponentIssuesOnly`.

10 Checklist Go/No-Go

— config serie creee et controlee,
— config globale mise a jour,
— mappings scripts a jour,
— mappings workflows a jour,
— phase Prepare validee,
— split composant valide,
— validation anti-doublons OK,
— upload dry-run OK,
— upload reel OK,
— datasource IDs traces dans `config_all_series.json`.

11 Points d'attention

— SHAs incorrects dans `linked_repo_refs` => non reproductibilite.
— serie declaree en config mais absente des `ValidateSet/seriesMap` => echec d'execution.
— serie absente des workflows => impossible a lancer via dispatch.
— upload composant sans split prealable => summary absent.